Programming Assignment 6—Register Allocation

Marc Lin

1 Implementation in RegAllocProfileSplit.cpp (Profile-based)

splitAllLiveRanges

For Live Range Splitting, we must implement for join points and fork points in CFG. In join point, when we calculate max predecessor frequency, we can iterate predecessors and get their frequency in *execCount[pred]*, use it to update *maxFreq*.

```
// calculate max predecessor frequency
uint64_t maxFreq = 0;
for (auto *pred : MBB.predecessors()) {
    maxFreq = std::max(maxFreq, execCount[pred]);
}
```

We use struct constructor to create new *SubrangeEdge* for virtual register live in specific BB as shown below.

```
// create one SubrangeEdge for every non-debug virtual register whose live interval
for (auto *LI : vregs) {    // LiveInterval *
    if (LIS->isLiveInToMBB(*LI, &MBB)) {
        SubrangeEdge e = SubrangeEdge{
            .origReg=LI->reg,
            .BB_dst=&MBB,
            .newReg=0, // 0 for the not-yet-created new register
            .maxPredFreq=maxFreq
        };
        insertedEdges.push_back(e);
    }
}
```

```
for (auto *succ : MBB.successors()) {
    if (succ->pred_size() < 2) {    // prevents double enumeration
        for (auto *LI : vregs) {
            if (LIS->isLiveInToMBB(*LI, succ)) {
                SubrangeEdge e = SubrangeEdge{
                    .origReg=LI->reg,
                    .BB_dst=succ,
                    .newReg=0, // 0 for the not-yet-created new register
                    .maxPredFreq=execCount[&MBB]  // MBB's frequency is simply fork
                };
                insertedEdges.push_back(e);
            }
        }
    }
}
```

deploySplits

When iterating edge in *insertedEdges*, we must record both the original and new registers as affected live ranges. Therefore, I use a local set *affectedEdge* to store virtual register's number.

```
// rebuild LiveIntervals for the affected virtual registers
std::set<unsigned> affectedRegs;
```

```
// record both the original and new registers as affected live ranges
// so that their liveness information can be rebuilt later in one bulk pass
affectedRegs.insert(e.origReg);
affectedRegs.insert(e.newReg);
```

We also need to insert a machine-level COPY instruction by using *BuildMI*, the usage I found is shown below. Since there is no TII in template code, I access it by using **MF->getTarget().getInstrInfo()*. After this, we register it with *LiveInterval.*

```
/* MachineInstrBuilder llvm::BuildMI(MachineBasicBlock &BB,
                                     MachineBasicBlock::iterator I,
                                     const DebugLoc &DL, const MCInstrDesc &MCID,
                                     bool IsIndirect, Register Reg,
                                     const MDNode *Variable, const MDNode *Expr)
*/
const TargetInstrInfo &TII = *MF->getTarget().getInstrInfo();
MachineInstr *MI = BuildMI(
    *pred,
    pred->getFirstTerminator(),
    DebugLoc(),
    TII.get(TargetOpcode::COPY),
    newReg
).addReg(e.origReg);

// register instruction with LiveIntervals
LIS->InsertMachineInstrInMaps(MI);
```

I use *MachineDominatorTree* to check dominance, simply as same as what we do in *DominatorTree*.

```
// rewrite register operands in machine instructions whose parent basic blocks are domina
MachineDominatorTree *MDT = &getAnalysis<MachineDominatorTree>();
for (auto &MBB : *MF) {
    if (MDT->dominates(e.BB_dst, &MBB)) {
        for (auto &MI : MBB) {
            for (auto &op : MI.operands()) {
                if (op.isReg() && op.getReg() == e.origReg) {
                    op.setReg(e.newReg);
                }
            }
        }
    }
}
```

2 Implementation in RegAllocGC.cpp (Graph Coloring)

In Chaitin's Algorithm, every time we select a node in IG, whose degree is lower than K (K-colorable), then remove it from the IG and put it into stack for coloring. If there is no node that has lower degree, we spill the node with the minimum weight (i.e. spill cost). After we have no node need to be spilled, we finally color the nodes in stack.

initGraph

When we follow the pseudo code, remember to initialize empty set for every virtual register even its live interval is not overlap with anyone. Also we have to add vertex in both src node and dest node's neighbor set.

```
// get the respective LiveInterval
LiveInterval *VirtReg = &LIS->getInterval(Reg);
vRegs.insert(VirtReg);
interferenceGraph[VirtReg] = std::set<LiveInterval *>();
```

```
// if two vregs' live ranges are overlapping, connect them by edge to avoid same register alloc
for (auto *vreg1 : vRegs) {
  for (auto *vreg2 : vRegs) {
    if (vreg1 == vreg2) continue;
    if (vreg1->overlaps(*vreg2)) {
      // connected by edge
      interferenceGraph[vreg1].insert(vreg2);
      interferenceGraph[vreg2].insert(vreg1);
    }
  }
}
```

simplifyGraph

In step 1, I use a special structure to iterate. First we find the smallest register number as *candidate*, which will be normally removed from IG. If we can find any *candidate*, then we don't have to move on to step 2. So in the block *if(candidate)*, the last statement is *continue;* , which helps us iterate the graph nodes and remove it normally without spill.

```
// victim to be removed from IG
if (degree < K) {
  // find the smallest register number as victim
  if (!candidate || vertex->reg < candidate->reg) {
    candidate = vertex;
  }
}
```

```
// if there is any candidate can be selected as colorable
if (candidate) {
  unsigned int numOfNeighbors = numOverlappingPhysRegs(candidate, rc) + interferenceGraph[can

  // remove condidate from all neighbors
  for (auto *neighbor : interferenceGraph[candidate]) {
    interferenceGraph[neighbor].erase(candidate);
  }
  interferenceGraph.erase(candidate);

  llvm::errs() << "Found neighbors = " << numOfNeighbors << " for " << *candidate << "\n";
  llvm::errs() << "Removal: " << *candidate << "\n";
  enqueue(candidate);

  continue;    // if find any candidate, we only iterate step 1 (because no need to spill)
}
```

In step 2, the implementation structure is similar to step 1. We choose the smallest register number as *victim*, and if there is any victim, we removed it as a usual node but inserted it in markForSpill (which is the member added by me, in the class *RAUSCC*). This will be used in step 3.

```
for (auto const& p : interferenceGraph) {
  LiveInterval *v = p.first;
  float currentWeight = v->weight;  // built-in member weight = spill cost
  if (!victim || currentWeight < minWeight || (currentWeight == minWeight && v->reg < victim->reg)) {
    victim = v;
    minWeight = currentWeight;
  }
}
```

```
markForSpill.insert(victim);
enqueue(victim);
```

In step 3, we allocate and see if physical register is available for virtual register. Iterate through *markForSpill* and update the neighbors with new *LiveInterval*. If it is a new interval and it is overlap with some existing interval, they will be connected by edge in the IG.

```
for (unsigned vreg : splitVRegs) {
  LiveInterval *newli = &LIS->getInterval(vreg);

  interferenceGraph[newli] = std::set<LiveInterval *>();
  for (auto &p : interferenceGraph) {
    LiveInterval *existingV = p.first;
    // update neighbors
    if (existingV != newli && newli->overlaps(*existingV)) {
      interferenceGraph[existingV].insert(newli);
      interferenceGraph[newli].insert(existingV);
    }
  }

  llvm::errs() << "add new interval for spilling to the IG: " << *newli << "\n";
}
```

allocation

According to the hint, we create *AllocationOrder* instance *Order* and iterate physical registers by *Order.next()*. If there is a free register with no interference, we can allocate virtual register to this physical register.

```
void RAUSCC::allocation() {
  // PA6:
  LiveInterval *virtReg = dequeue();
  while (virtReg) {
    AllocationOrder Order(virtReg->reg, *VRM, RegClassInfo);
    while (unsigned PhysReg = Order.next()) {
      // Check for interference in PhysReg
      if (Matrix->checkInterference(*virtReg, PhysReg) == LiveRegMatrix::IK_Free) {
        llvm::errs() << "Assigning to physical register " << TRI->getName(PhysReg) << ": " << *vi
        Matrix->assign(*virtReg, PhysReg);
        break;
      }
    }
    virtReg = dequeue();
  }
}
```